Unit-2 Requirement Analysis

Software Requirement Tasks (Requirements Capturing):

Requirements engineering-

Requirements engineering tasks involve a structured process to identify, analyze, and manage the needs and constraints of a software project. This ensures that the final product meets user expectations and functions correctly.

What is Requirement Engineering?

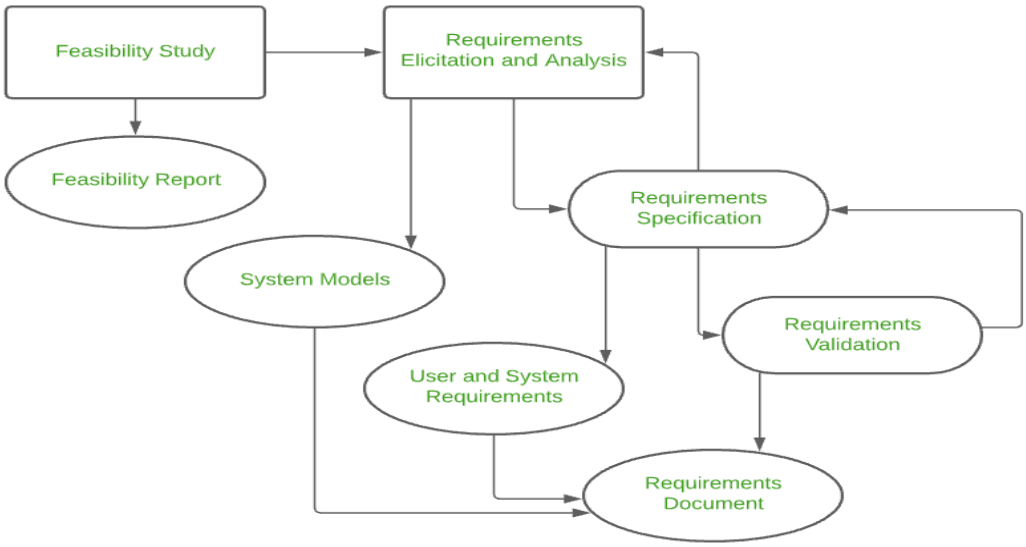
Requirements engineering is a broad domain that focuses on being the connector between modelling, analysis, design, and construction. It is the process that defines, identifies, manages, and develops requirements in a software engineering design process. This process uses tools, methods, and principles to describe the system's behaviour and the constraints that come along with it.

Requirements engineering is the most important part every business must follow, in order to build and release a project successfully, as it is the foundation to key planning and implementation.

Requirement Engineering Process

Following are the Requirement Engineering Process

1. Feasibility Study
2. Requirements elicitation
3. Requirements specification
4. Requirements for verification and validation
5. Requirements management



Requirements Engineering Tasks

The software requirements engineering process includes the following steps of activities:

1. Inception
2. Elicitation
3. Elaboration
4. Negotiation
5. Specification
6. Validation
7. Requirements Management

Let's discuss each of these steps in detail.

1. Inception

This is the first phase of the requirements analysis process. This phase gives an outline of how to get started on a project. In the inception phase, all the basic questions are asked on how to go about a task or the steps required to accomplish a task. A basic understanding of the problem is gained and the nature of the solution is addressed. Effective communication is very important in this stage, as this phase is the foundation as to what has to be done further. Overall in the inception phase, the following criteria have to be addressed by the software engineers:

- Understanding of the problem.
- The people who want a solution.
- Nature of the solution.
- Communication and collaboration between the customer and developer.

2. Elicitation

This is the second phase of the requirements analysis process. This phase focuses on gathering the requirements from the stakeholders. One should be careful in this phase, as the requirements are what establishes the key purpose of a project. Understanding the kind of requirements needed from the customer is very crucial for a developer. In this process, mistakes can happen in regard to, not implementing the right requirements or forgetting a part. The right people must be involved in this phase. The following problems can occur in the elicitation phase:

- **Problem of Scope:** The requirements given are of unnecessary detail, ill-defined, or not possible to implement.
- **Problem of Understanding:** Not having a clear-cut understanding between the developer and customer when putting out the requirements needed. Sometimes the customer might not know what they want or the developer might misunderstand one requirement for another.
- **Problem of Volatility:** Requirements changing over time can cause difficulty in leading a project. It can lead to loss and wastage of resources and time.

3. Elaboration

This is the third phase of the requirements analysis process. This phase is the result of the inception and elicitation phase. In the elaboration process, it takes the requirements that have been stated and gathered in the first two phases and refines them. Expansion and looking into it further are done as well. The main task in this phase is to indulge in modelling activities and develop a prototype that elaborates on the features and constraints using the necessary tools and functions.

4. Negotiation

This is the fourth phase of the requirements analysis process. This phase emphasizes discussion and exchanging conversation on what is needed and what is to be eliminated. In the negotiation phase, negotiation is between the developer and the customer and they dwell on how to go about the project with limited business resources. Customers are asked to prioritize the requirements and make guesstimates on the conflicts that may arise along with it. Risks of all the requirements are taken into consideration and negotiated in a way where the customer and developer are both satisfied with reference to the further implementation.

The following are discussed in the negotiation phase:

- Availability of Resources.
- Delivery Time.
- Scope of requirements.
- Project Cost.
- Estimations on development.

5. Specification

This is the fifth phase of the requirements analysis process. This phase specifies the following:

- Written document.
- A set of models.
- A collection of use cases.

- A prototype.

In the specification phase, the requirements engineer gathers all the requirements and develops a working model. This final working product will be the basis of any functions, features or constraints to be observed. The models used in this phase include ER (Entity Relationship) diagrams, DFD (Data Flow Diagram), FDD (Function Decomposition Diagrams), and Data Dictionaries.

A software specification document is submitted to the customer in a language that he/she will understand, to give a glimpse of the working model.

6. Validation

This is the sixth phase of the requirements analysis process. This phase focuses on checking for errors and debugging. In the validation phase, the developer scans the specification document and checks for the following:

- All the requirements have been stated and met correctly
- Errors have been debugged and corrected.
- Work product is built according to the standards.

This requirements validation mechanism is known as the formal technical review. The review team that works together and validates the requirements include software engineers, customers, users, and other stakeholders. Everyone in this team takes part in checking the specification by examining for any errors, missing information, or anything that has to be added or checking for any unrealistic and problematic errors. Some of the validation techniques are the following-

- Requirements reviews/inspections.
- Prototyping.
- Test-case generation.
- Automated consistency analysis.

7. Requirements Management

This is the last phase of the requirements analysis process. Requirements management is a set of activities where the entire team takes part in identifying, controlling, tracking, and establishing the requirements for the successful and smooth implementation of the project. In this phase, the team is responsible for managing any changes that may occur during the project. New requirements emerge, and it is in this phase, responsibility should be taken to manage and prioritize as to where its position is in the project and how this new change will affect the overall system, and how to address and deal with the change. Based on this phase, the working model will be analysed carefully and ready to be delivered to the customer.

Prioritizing Requirements

Prioritization in product management is the practice of deciding which features, tasks, or initiatives to work on first. It involves evaluating items based on impact, value, urgency, and effort. Effective prioritization ensures that teams focus their time and resources on the work that delivers the greatest value to users and the business. It also helps product managers manage trade-offs and maintain clarity in decision-making.

What are Product Prioritization Frameworks

Product prioritization frameworks are structured approaches used by product managers and teams to decide which features, initiatives, or tasks should be worked on first. These frameworks help teams allocate time and resources effectively by evaluating work based on factors such as customer value, business impact, effort, and strategic goals.

Using the right prioritization framework helps teams answer critical questions, such as:

- Are we focusing on the highest-value initiatives for the business?
- Are we delivering meaningful value to our customers?
- Does our work align with overall business objectives?
- Can we realistically and successfully bring this product to market?

Common Product Prioritization Frameworks

Product managers use various prioritization frameworks to decide which features or initiatives should be worked on first. These frameworks help balance customer value, business impact, effort, and urgency.

Some commonly used product prioritization frameworks include:

- MoSCoW Method
- Kano Model
- RICE Score
- WSJF (Weighted Shortest Job First)
- Value vs. Effort Matrix
- Eisenhower Matrix

Let us see about each framework in detail:

Kano Model:

Introduction:

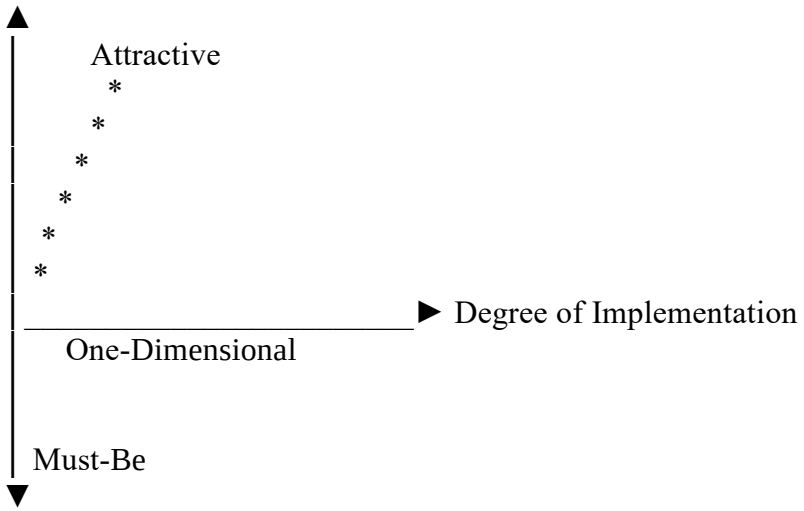
Requirements prioritization is the process of ranking system features based on their importance and impact on customer satisfaction. The **Kano Model** is a popular technique used to classify requirements into categories according to how they influence user satisfaction.

The Kano model helps us understand what features will really make customers happy. In Image provided below. On the horizontal line, we have three types of features:

1. **Must-haves (Basic Features):** These are things your product must have. If you don't have them, people won't even consider buying your product.
2. **Performance Features:** The more you work on these, the happier customers will be. It's like adding extra goodness to your product.
3. **Delighters (Excitement Features):** These are cool surprises. Customers don't expect them, but when they get them, it makes them super happy.

- On the vertical line, we measure customer satisfaction. At the bottom, it's like the product didn't meet their needs at all. As you go up, satisfaction increases until it's fully met at the top.
- To understand what people want, you ask them questions using a Kano questionnaire. You figure out how they feel about having or not having specific features.

Here's the key idea: The more you focus on improving features in each of these categories, the happier your customers will be. So, it's about wisely spending your time and resources to create a product that really satisfies your customers.



Features of Kano Model

Basic Features:

- Essential features that customers expect by default. Their absence causes dissatisfaction, but their presence does not significantly increase satisfaction
- Examples: A smartphone having a battery, a car having four wheels.

Performance Features:

- Features that directly influence customer satisfaction and are used to compare products. Better performance leads to higher satisfaction.
- Examples: Fuel efficiency in a car, camera quality in a smartphone.

Excitement Features:

- Unexpected features that delight customers and create a strong competitive advantage. They are not essential but can significantly enhance user experience.
- Examples: Facial recognition, built-in navigation systems.

Use Case of Kano Model

- Used to prioritize features based on their impact on customer satisfaction and delight.
- Helps distinguish between essential features, differentiators, and delight factors.

Benefits of Kano Model

- **Customer-Centric:** Focuses on understanding customer needs and expectations.
- **Effective Prioritization:** Helps prioritize features that drive satisfaction and differentiation.
- **Competitive Advantage:** Identifies features that can delight users and set the product apart.

Drawbacks of Kano Model

1. **Subjectivity**: The Kano Model is subjective and relies on the judgment of the product manager and team members to categorize features.
2. **Complexity**: The Kano Model can be complex and difficult to apply in practice, especially for products with a large number of features.
3. **Limited Scope**: The Kano Model is limited in scope and does not take into account other factors that may affect prioritization, such as cost, effort, or risk.

Real-Life Case Study — Kano Model Prioritization

Example: Online Food Delivery Application

A company plans to develop an online food delivery app. Different features are prioritized using the Kano Model based on customer satisfaction.

✓ Must-Be Requirements (Basic Needs)

These are essential features expected by every user.

- User registration and login
- Browsing restaurants and menu
- Placing orders
- Secure online payment

- Order confirmation

☞ If these are missing, customers will not use the app.

✓ One-Dimensional Requirements (Performance Needs)

Customer satisfaction increases as performance improves.

- Faster delivery time
- Accurate real-time order tracking
- Quick app response
- Reliable service

☞ Better performance leads to higher satisfaction.

✓ Attractive Requirements (Excitement Needs)

Unexpected features that delight users.

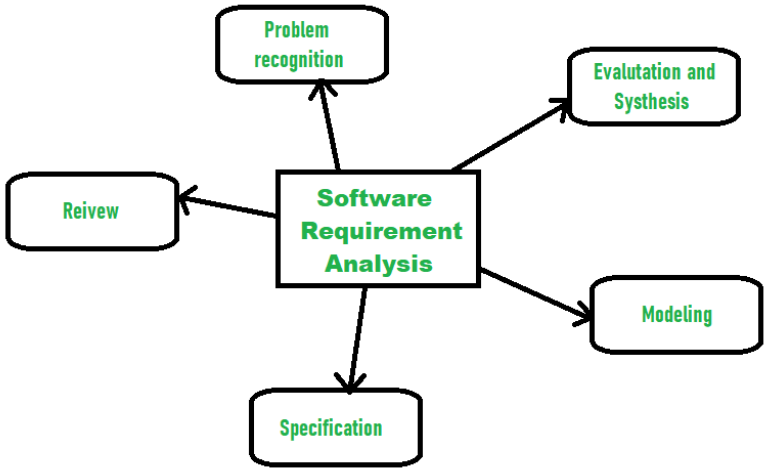
- Discount coupons and special offers
- Personalized food recommendations
- "Reorder previous order" option
- Voice search for food items

☞ These features attract more users but are not mandatory.

Software Requirement Analysis:

Software requirement means requirement that is needed by software to increase quality of software product. These requirements are generally a type of expectation of user from software product that is important and need to be fulfilled by software. Analysis means to examine something in an organized and specific manner to know complete details about it.

Therefore, **Software requirement analysis** simply means complete study, analyzing, describing software requirements so that requirements that are genuine and needed can be fulfilled to solve problem. There are several activities involved in analyzing Software requirements. Some of them are given below:



1. **Problem Recognition:**
   The main aim of requirement analysis is to fully understand main objective of requirement that includes why it is needed, does it add value to product, will it be beneficial, does it increase quality of the project, and does it will have any other effect.
2. **Evaluation and Synthesis:**
   Evaluation means judgement about something whether it is worth or not and synthesis means to create or form something.
3. **Modeling:**
   After complete gathering of information from above tasks, functional and behavioural models are established after checking function and behaviour of system using a domain model that also known as the conceptual model.

4. **Specification:**
   The software requirement specification (SRS) which means to specify the requirement whether it is functional or non-functional should be developed.
5. **Review:**
   After developing the SRS, it must be reviewed to check whether it can be improved or not and must be refined to make it better and increase the quality.

Scenario-Based Modeling

Scenario-based modeling is a technique used in various fields, including business, finance, and software development, to analyze and plan for different potential situations or scenarios. It involves creating models or simulations based on specific scenarios to understand their potential impact and make informed decisions.

Uses of Scenario-Based Modeling

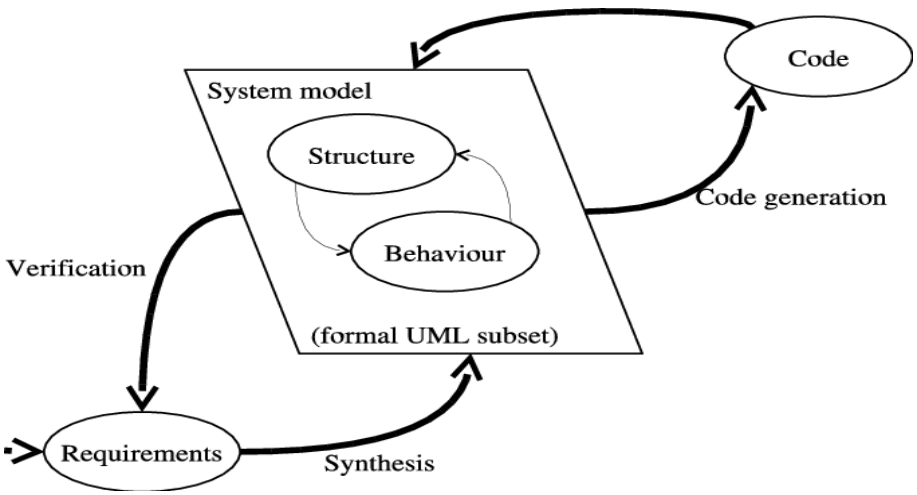
- Risk Management: It helps in assessing and mitigating risks by simulating various risk scenarios and their potential outcomes.
- Financial Planning: It aids in financial forecasting by modeling different economic or market scenarios.
- Strategic Planning: Organizations use scenario-based modeling to develop strategic plans based on different future scenarios.
- Software Development: In software engineering, scenario-based modeling is used to create use case scenarios for system design and testing.

Process of Scenario-Based Modeling

- Identify Scenarios: Determine the different potential situations or events that could impact the system or process.
- Create Models: Develop models or simulations based on each scenario, considering relevant variables and factors.
- Analyze Outcomes: Run the models to analyze the outcomes and implications of each scenario.
- Decision Making: Use the insights gained from the scenario-based modeling to make informed decisions and plans.

Benefits of Scenario-Based Modeling

- **Risk Assessment**: It helps in identifying and understanding potential risks and their impact.
- **Strategic Planning**: Enables organizations to develop robust strategies based on various future possibilities.
- **Improved Decision Making**: Provides a basis for making informed decisions by considering multiple potential outcomes.



Example

In financial planning, scenario-based modeling can be used to simulate different economic conditions (e.g., recession, inflation, or stable growth) to understand the impact on investment portfolios and make appropriate adjustments.

By employing scenario-based modeling, organizations and individuals can better prepare for the uncertainties of the future and make proactive decisions.

Unified Modeling Language (UML) Diagrams

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. UML is not a programming language, it is rather a visual language.

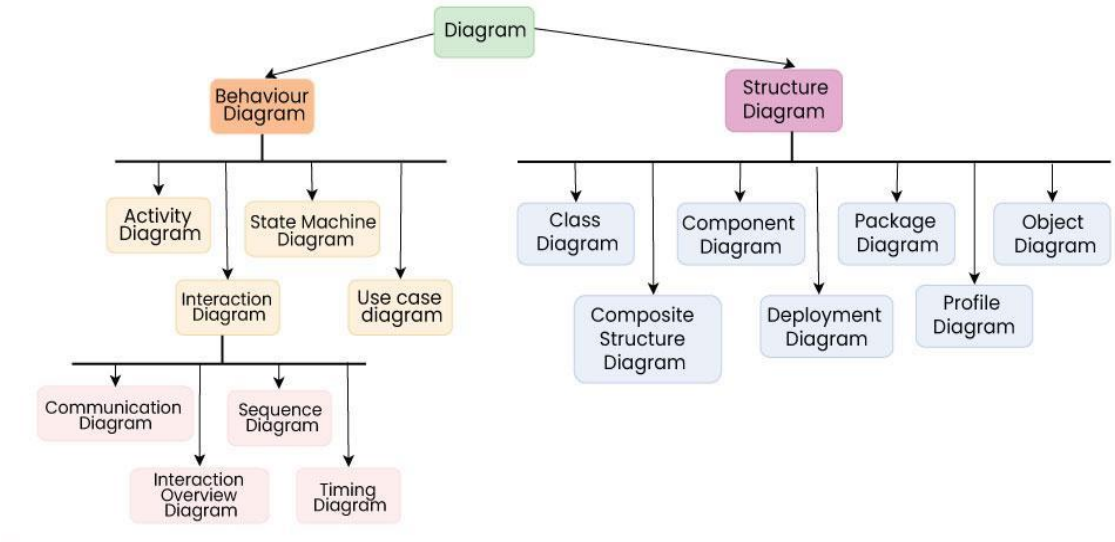
- We use UML diagrams to show the behavior and structure of a system.
- UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.

Need of UML

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non-programmers about essential requirements, functionalities, and processes of the system.
- A lot of time is saved down the line when teams can visualize processes, user interactions, and the static structure of the system.

Types of UML Diagrams

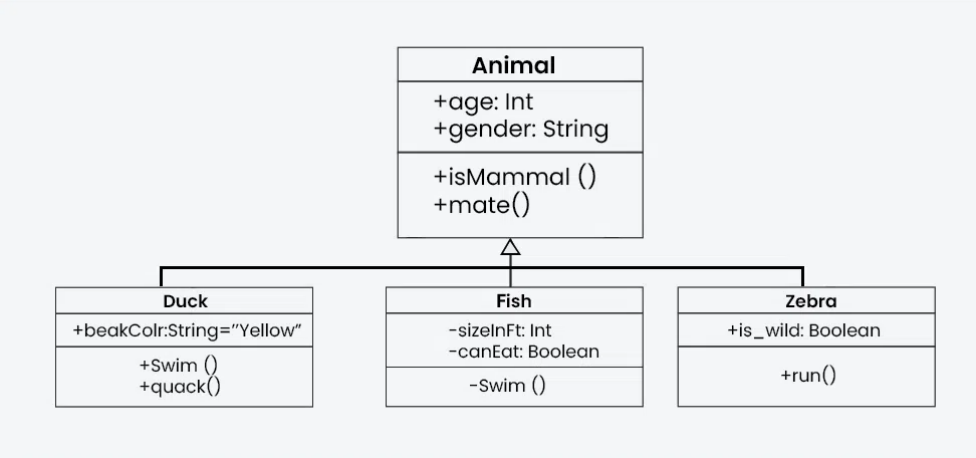
UML is linked with object-oriented design and analysis. UML makes use of elements and forms associations between them to form diagrams. Diagrams in UML can be broadly classified as:



Structural UML diagrams are visual representations that depict the static aspects of a system, including its classes, objects, components, and their relationships, providing a clear view of the system's architecture. Structural UML diagrams include the following types:

Class Diagram

The most widely use UML diagram is the class diagram. It is the building block of all object oriented software systems. We use class diagrams to depict the static structure of a system by showing system's classes, their methods and attributes. Class diagrams also help us identify relationship between different classes or objects.



Composite Structure Diagram

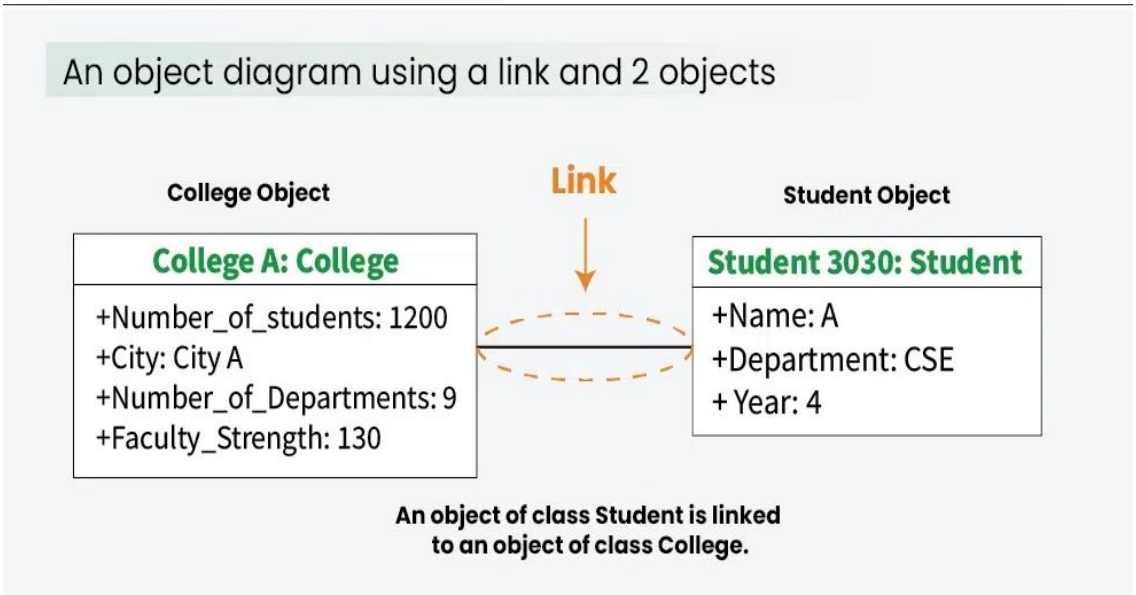
We use composite structure diagrams to represent the internal structure of a class and its interaction points with other parts of the system.

- A composite structure diagram represents relationship between parts and their configuration which determine how the classifier (class, a component, or a deployment node) behaves.
- They represent internal structure of a structured classifier making the use of parts, ports, and connectors.
- They are similar to class diagrams except they represent individual parts in detail as compared to the entire class.

Object Diagram

An Object Diagram can be referred to as a screenshot of the instances in a system and the relationship that exists between them. Since object diagrams depict behaviour when objects have been instantiated, we are able to study the behaviour of the system at a particular instant.

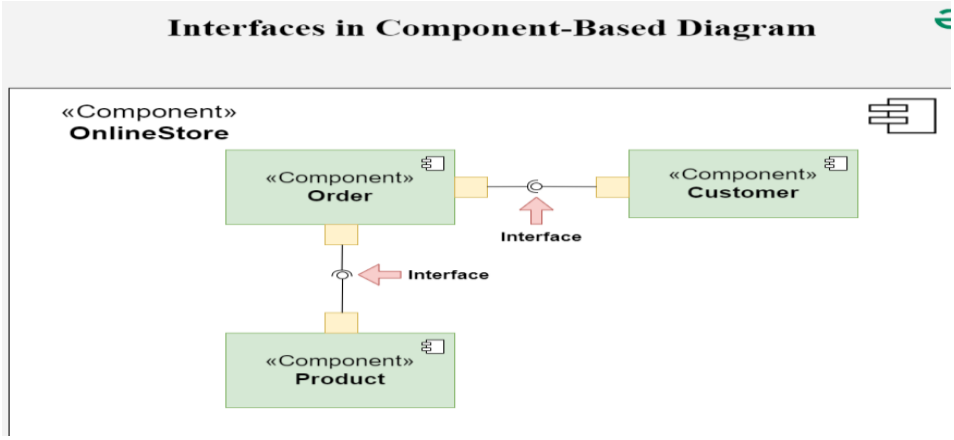
- An object diagram is similar to a class diagram except it shows the instances of classes in the system.
- We depict actual classifiers and their relationships making the use of class diagrams.
- On the other hand, an Object Diagram represents specific instances of classes and relationships between them at a point of time.



Component Diagram

Component diagrams are used to represent how the physical components in a system have been organized. We use them for modelling implementation details.

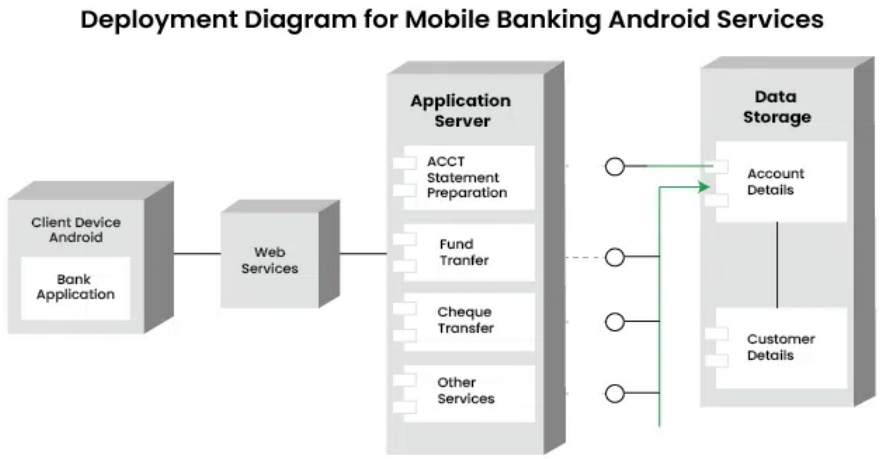
- Component Diagrams depict the structural relationship between software system elements and help us in understanding if functional requirements have been covered by planned development.
- Component Diagrams become essential to use when we design and build complex systems.
- Interfaces are used by components of the system to communicate with each other.



Deployment Diagram

Deployment Diagrams are used to represent system hardware and its software. It tells us what hardware components exist and what software components run on them.

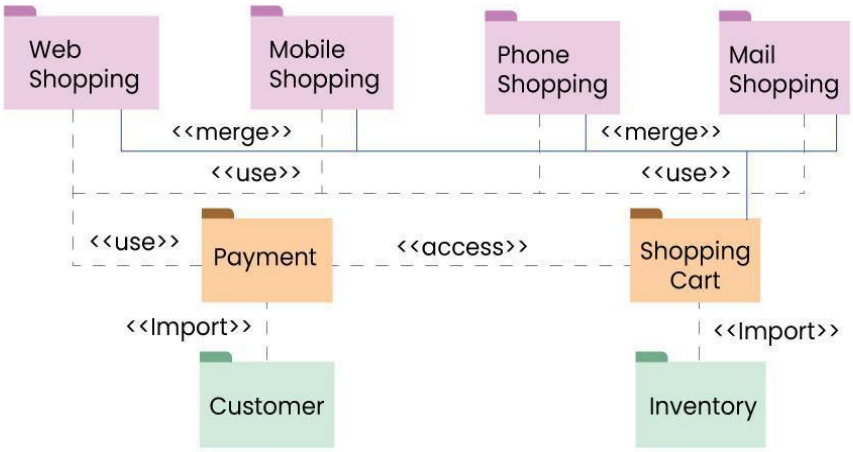
- We illustrate system architecture as distribution of software artifacts over distributed targets.
- An artifact is the information that is generated by system software.
- They are primarily used when a software is being used, distributed or deployed over multiple machines with different configurations.



Package Diagram

We use Package Diagrams to depict how packages and their elements have been organized. A package diagram simply shows us the dependencies between different packages and internal composition of packages.

- Packages help us to organize UML diagrams into meaningful groups and make the diagram easy to understand.
- They are primarily used to organize class and use case diagrams.



Behavioural UML Diagrams

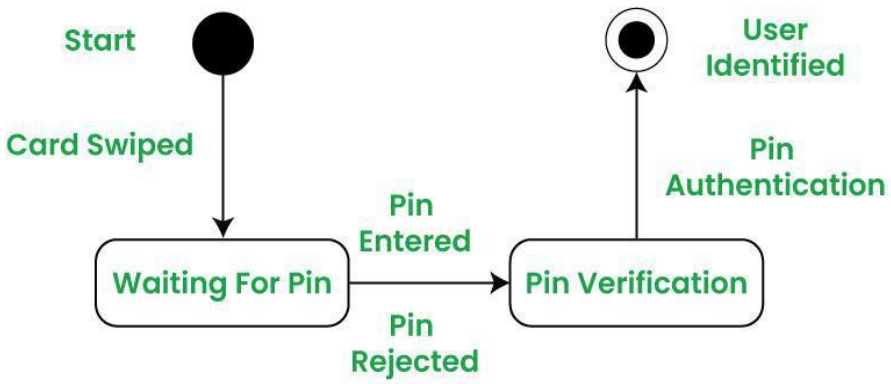
Behavioural UML diagrams are visual representations that depict the dynamic aspects of a system, illustrating how objects interact and behave over time in response to events.

State Machine Diagrams

A state diagram is used to represent the condition of the system or part of the system at finite instances of time. It's a behavioral diagram and it represents the behavior using finite state transitions.

- State diagrams are also referred to as **State machines** and **State-chart Diagrams**
- These terms are often used interchangeably. So simply, a state diagram is used to model the dynamic behavior of a class in response to time and changing external stimuli.

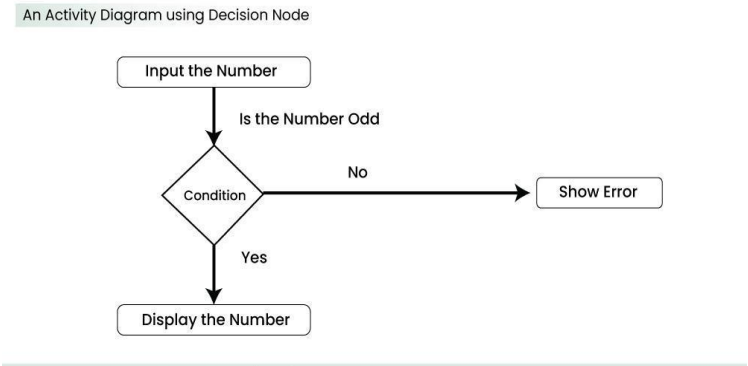
A State Machine Diagram for user verification



Activity Diagrams

We use Activity Diagrams to illustrate the flow of control in a system. We can also use an activity diagram to refer to the steps involved in the execution of a use case.

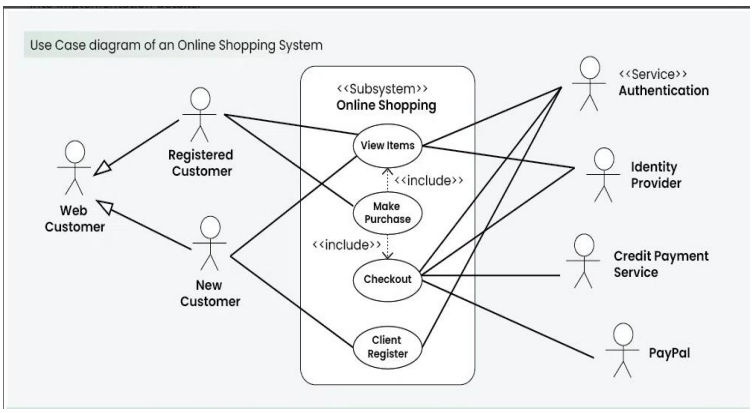
- We model sequential and concurrent activities using activity diagrams. So, we basically depict workflows visually using an activity diagram.
- An activity diagram focuses on condition of flow and the sequence in which it happens.
- We describe or depict what causes a particular event using an activity diagram.



Use Case Diagrams

Use Case Diagrams are used to depict the functionality of a system or a part of a system. They are widely used to illustrate the functional requirements of the system and its interaction with external agents (actors).

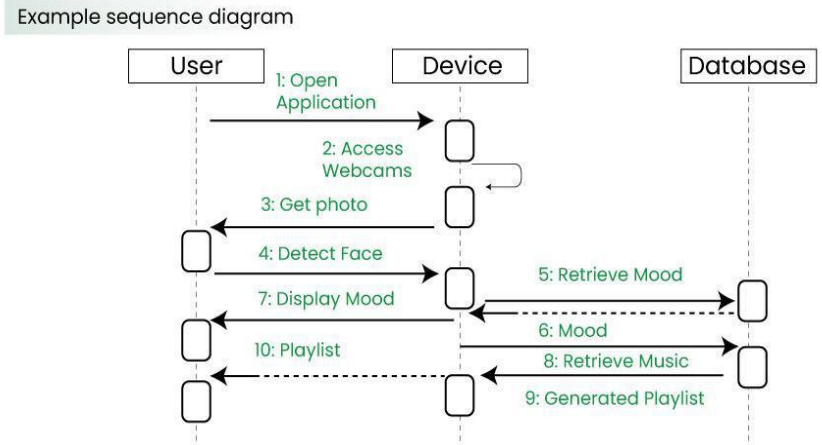
- A use case is basically a diagram representing different scenarios where the system can be used.
- A use case diagram gives us a high level view of what the system or a part of the system does without going into implementation details.



Sequence Diagram

A sequence diagram simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place.

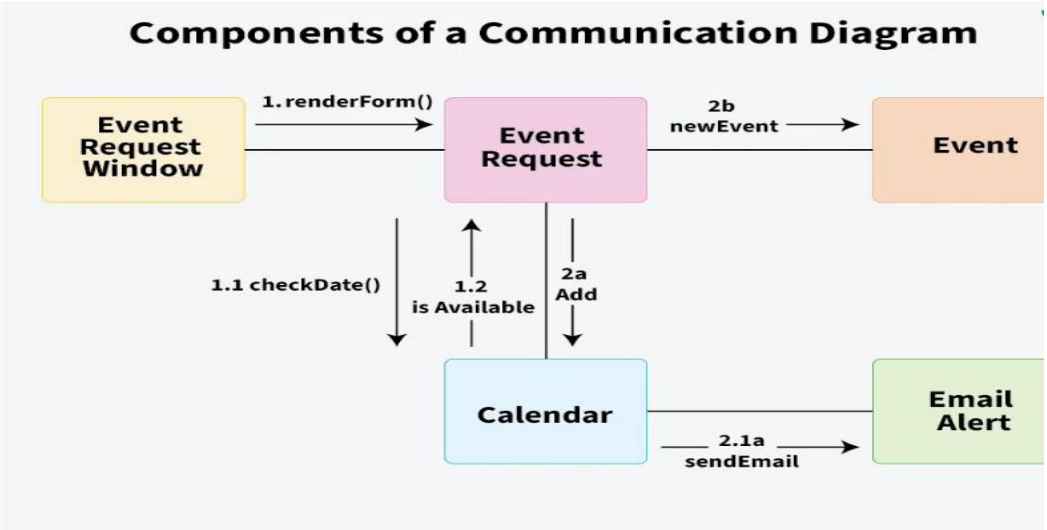
- We can also use the terms event diagrams or event scenarios to refer to a sequence diagram.
- Sequence diagrams describe how and in what order the objects in a system function.
- These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.



Communication Diagram

A Communication Diagram (known as Collaboration Diagram in UML 1.x) is used to show sequenced messages exchanged between objects.

- A communication diagram focuses primarily on objects and their relationships.
- We can represent similar information using Sequence diagrams, however communication diagrams represent objects and links in a free form.

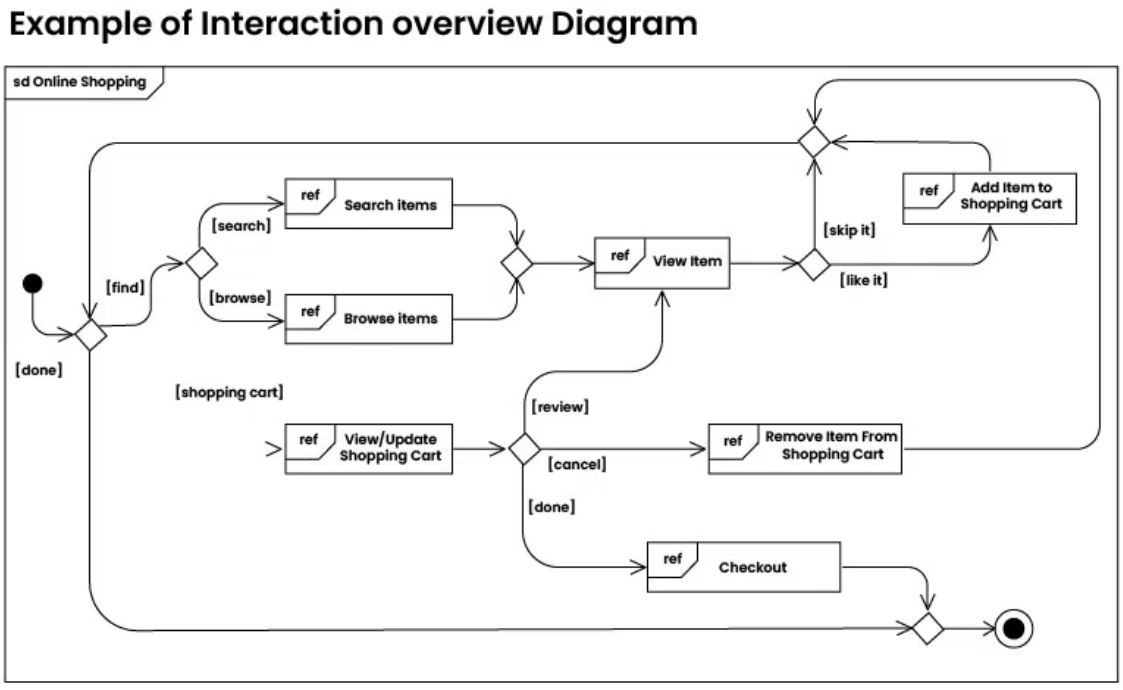


Timing Diagram

Timing Diagram are a special form of Sequence diagrams which are used to depict the behavior of objects over a time frame. We use them to show time and duration constraints which govern changes in states and behavior of objects.

Interaction Overview Diagram

An Interaction Overview Diagram (IOD) is a type of UML (Unified Modeling Language) diagram that illustrates the flow of interactions between various elements in a system or process. It provides a high-level overview of how interactions occur, including the sequence of actions, decisions, and interactions between different components or objects.



Additions in UML 2.0

- Software development methodologies like agile have been incorporated and scope of original UML specification has been broadened.
- Originally UML specified 9 diagrams. UML 2.x has increased the number of diagrams from 9 to 13. The four diagrams that were added are: timing diagram, communication diagram, interaction overview diagram and composite structure diagram. UML 2.x renamed state chart diagrams to state machine diagrams.
- UML 2.x added the ability to decompose software system into components and sub-components.

Tools for creating UML Diagrams

There are several tools available for creating Unified Modeling Language (UML) diagrams, which are commonly used in software development to visually represent system architecture, design, and implementation. Here are some popular UML diagram creating tools:

- **Lucidchart:** Lucidchart is a web-based diagramming tool that supports UML diagrams. It's user-friendly and collaborative, allowing multiple users to work on diagrams in real-time.
- **Draw.io:** Draw.io is a free, web-based diagramming tool that supports various diagram types, including UML. It integrates with various cloud storage services and can be used offline.
- **Visual Paradigm:** Visual Paradigm provides a comprehensive suite of tools for software development, including UML diagramming. It offers both online and desktop versions and supports a wide range of UML diagrams.
- **StarUML:** StarUML is an open-source UML modeling tool with a user-friendly interface. It supports the standard UML 2.x diagrams and allows users to customize and extend its functionality through plugins.

Data Modeling

Data modeling is the process of creating a structured and visual representation of data, defining how data elements relate to one another within a system. It helps translate business requirements into organized data structures that support accurate analysis and effective decision-making.

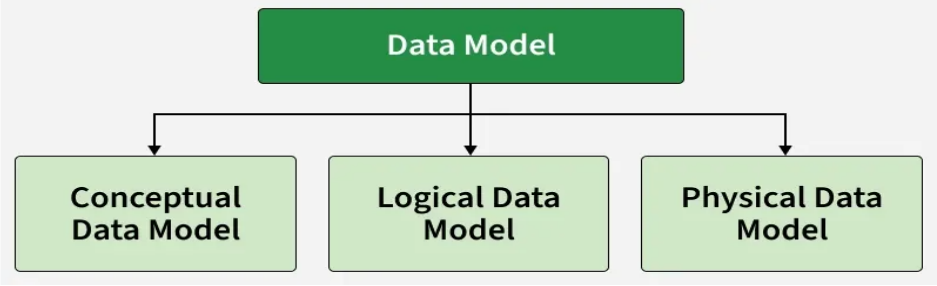
- Defines data structures, relationships and business rules within an organization
- Identifies and analyzes data requirements to support business processes
- Ensures data consistency, accuracy and integrity across systems
- Helps analysts understand relationships between different data entities
- Enables efficient data storage, retrieval and performance optimization
- Supports trend analysis and data-driven decision-making

Data Model

A data model is a visual and logical representation of an organization data elements and the relationships between them. It helps structure and organize data in alignment with business processes, enabling effective communication between business users and technical teams. Data models define how data is stored, accessed, shared and maintained across information systems.

Types of Data Models

There are three main types of data models:



1. Conceptual Data Model: The conceptual data model represents high-level, abstract business concepts and structures. It is primarily used in the early stages of a project to define and examine business requirements and relationships.

- **Purpose**: Organize and define business problems, rules and concepts.
- **Focus**: High-level overview of data such as customer data, market data and purchase data.
- **Audience**: Business stakeholders and analysts.
- **Example Use**: Understanding the relationship between customers and their orders before designing the database structure.

2. Logical Data Model: The logical data model builds upon the conceptual model and provides a detailed representation of the data at a logical level.

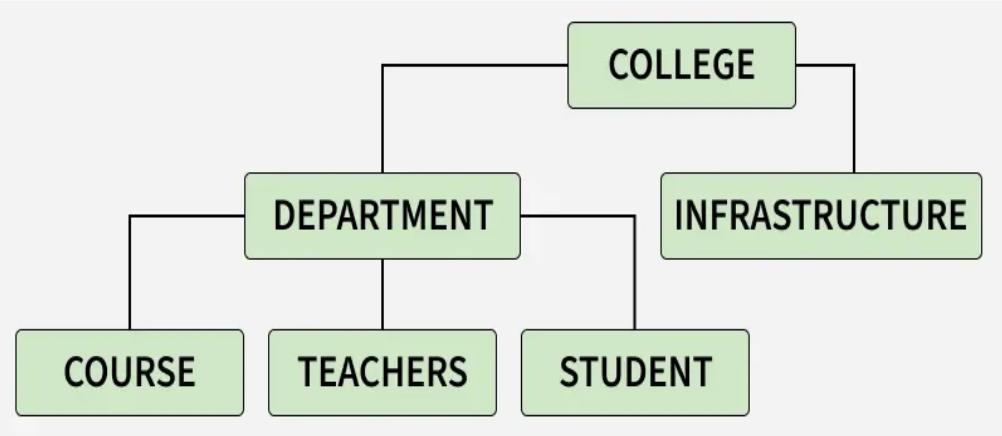
- **Purpose**: Define tables, columns, relationships and constraints that form the data structure.
- **Focus**: Structure of the data without depending on any specific database management system (DBMS).
- **Audience**: Data architects and analysts.
- **Example Use**: Outlining the schema, relationships and rules for customer and order data, which later guides the physical database design.

3. Physical Data Model: The physical data model represents the implementation of the data model in a specific database system.

- **Purpose**: Define every element needed to construct a database, including tables, columns, keys and constraints (primary key, foreign key, NOT NULL, etc.).
- **Focus**: Actual implementation of the database using queries and the chosen DBMS features.
- **Audience**: Developers and database administrators (DBAs).
- **Example Use**: Creating the database schema and ensuring that all constraints and relationships are enforced in the physical database.

Types of Data Modeling

1. Hierarchical Model: The structure of the hierarchical model resembles a tree. The remaining child nodes are arranged in a certain sequence and there is only one root node or alternatively one parent node. However the hierarchical approach is no longer widely applied. approach connections in the actual world may be modelled using this approach.



For example in a college there are many courses, many professors and students. So college became a parent and professors and students became its children.

2. Relational Model: Relational Mode represent the links between tables by representing data as rows and columns in tables. It is frequently utilised in database design and is strongly related to relational database management systems (RDBMS).

3. Object-Oriented Data Model: In this model data is represented as objects similar to those used in object-oriented programming, creating objects with stored values is the object-oriented method. In addition to allowing data abstraction, inheritance and encapsulation the object-oriented architecture facilitates communication.

4. Network Model: We have to represent objects and the relationships using the network model. One of its features is a schema, which is a graph representation of the data. An item is stored within a node and the relationship between them is represented as an edge. This allows them to generalise the maintenance of many parent and child records.

5. ER-Model: A high-level relational model called the entity-relationship model (ER model) is used to specify the data pieces and relationships between the entities in a system. This conceptual design gives us an easier-to-understand perspective on the facts. An entity-relationship diagram, which is made up of entities, attributes and relationships, is used in this model to depict the whole database.

A relationship between entities is called an association. Mapping cardinality many associations like:

- One to One
- One to Many
- Many to One
- Many to Many

Benefits of Data Modeling

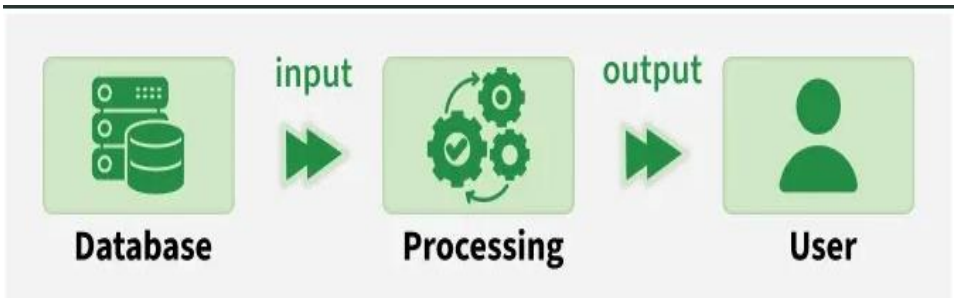
In order to organise and structure data and provide database design clarity, data modelling is essential. It acts as a common language, promoting efficient stakeholder communication. It directs the best database architecture for effective data storage and retrieval through visual representation.

1. Visualizes complex data structures, providing a clear roadmap for understanding relationships.
2. Acts as a universal language, fostering effective communication between business and technical stakeholders.
3. Creates organized databases by defining entities, properties and relationships.
4. Enhances data quality and integrity by reducing anomalies and redundancy through normalization.
5. Minimizes errors in database and application development.

What is DFD (Data Flow Diagram)

A Data Flow Diagram (DFD) is a graphical tool used to represent how data moves through a system. It shows data inputs, outputs, data stores, and the processes that transform the data. DFDs provide a high-level overview of system functionality and are widely used in structured analysis due to their simplicity and clarity for both technical and non-technical users.

DFDs help visualize the major steps in a system and illustrate how information flows between different components.



Characteristics of Data Flow Diagram

1. Graphical Representation: DFDs use standard symbols (processes, data stores, data flows, and external entities) to simplify complex systems into easy-to-read diagrams.
2. Problem Analysis: They help analyze system requirements by clarifying how data is processed, stored, and transferred.

3. Abstraction: DFDs focus on what the system does, not how it is implemented. Technical details are intentionally omitted.
4. Hierarchy: DFDs are built in levels: Level 0 (Context Diagram) → High-level overview and Level 1+ → More detailed breakdown of system processes

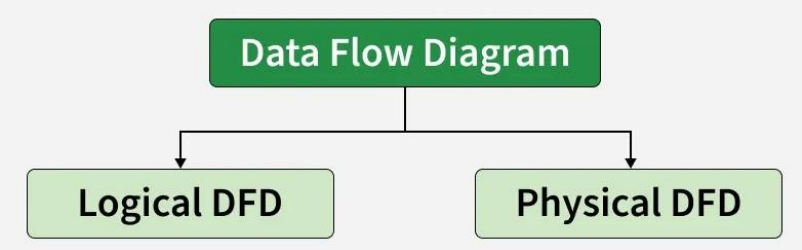
Levels of Data Flow Diagram

DFDs are categorized into various levels, with each level providing different degrees of detail. The levels are numbered from **0** and onward. The higher the level, the more detailed the diagram becomes.

- **Level 0 DFD** (Context Diagram): Represents the entire system as one single process, showing its interaction with external entities. It highlights major inputs and outputs without internal details.
- **Level 1 DFD**: Breaks the Level 0 process into major subprocesses. Shows internal data flows and data stores.
- **Level 2 and Beyond**: Further decomposes Level 1 subprocesses for a detailed view of specific functional areas. Useful for large or complex systems requiring deeper analysis.

Types of Data Flow Diagram

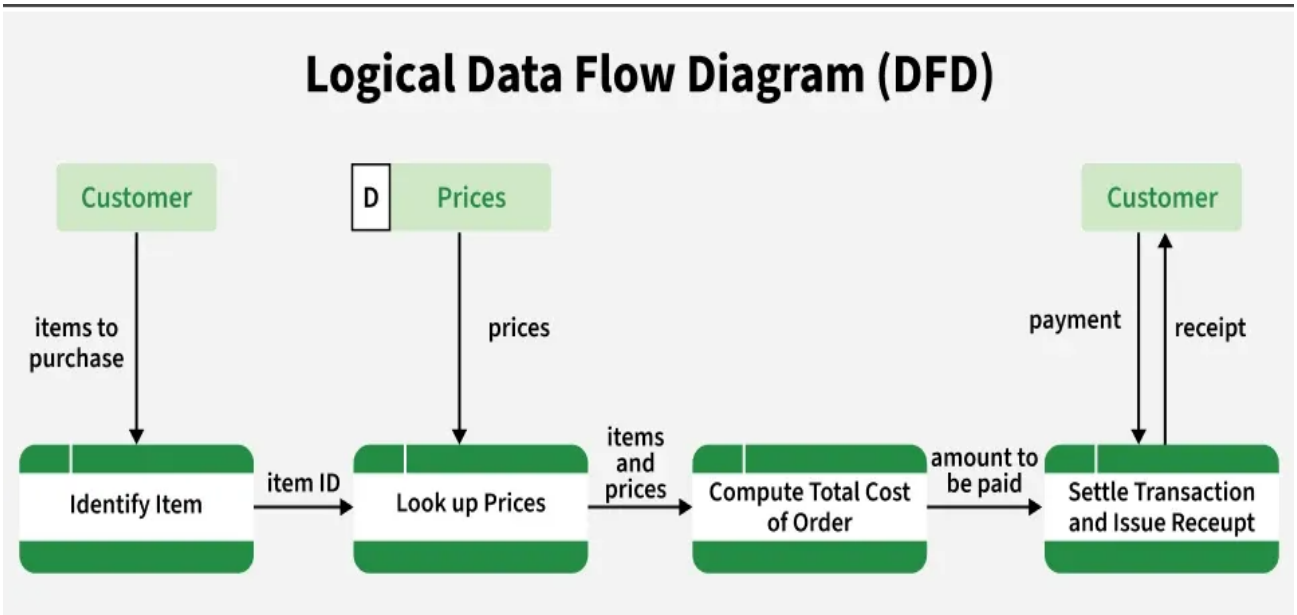
DFDs can be classified into two main types, each focusing on a different perspective of system design:



1. Logical DFD

The Logical Data Flow Diagram mainly focuses on the system process. It illustrates how data flows in the system. Logical Data Flow Diagram (DFD) mainly focuses on high level processes and data flow without diving deep into technical implementation details.

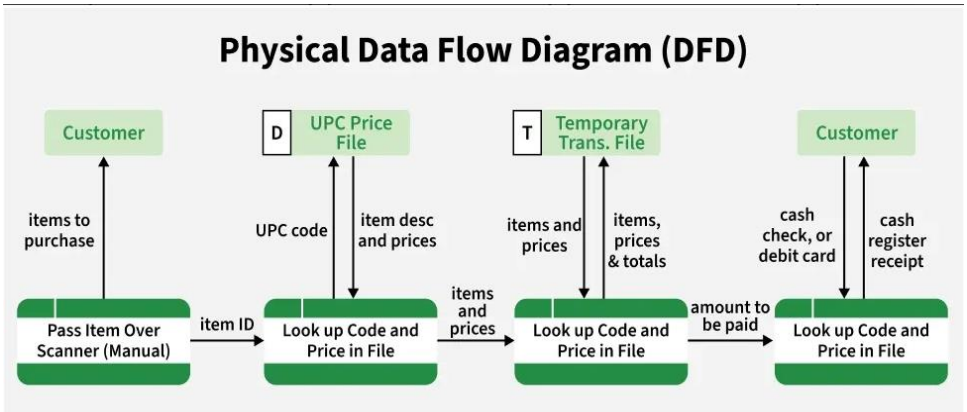
Logical DFDs is used in various organizations for the smooth running of system. Like in a Banking software system, it is used to describe how data is moved from one entity to another.



2. Physical DFD

Physical data flow diagram shows how the data flow is actually implemented in the system. In the Physical Data Flow Diagram (DFD), we include additional details such as data storage, data transmission, and specific technology or system components including:

- Hardware
- Software
- Databases
- File structures
- Actual data movement and storage mechanisms



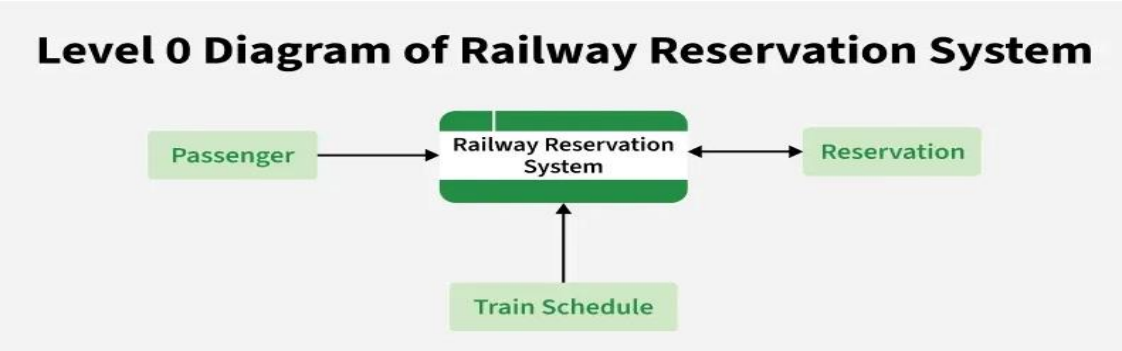
Example: Levels of DFD (Railway Reservation System)

To demonstrate how DFD levels are created, let's take an example of a Railway Reservation System. This example helps illustrate how data moves between users, processes, and data stores—such as passenger information, booking records, schedules, and payments - at different levels of detail. Each level of the DFD breaks the system into more specific processes to show how the reservation workflow operates internally.

Level 0 DFD

Shows the reservation system as one process interacting with:

- Passenger (external entity)
- Payment system
- Database (data store)

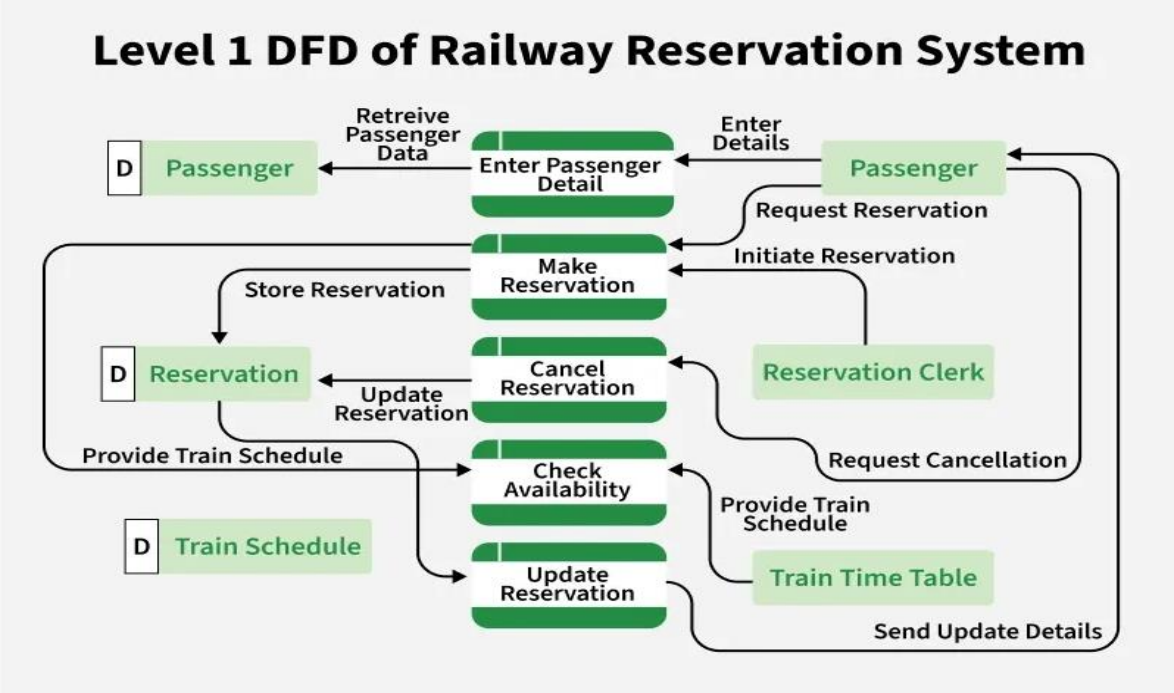


Level 1 DFD

Breaks the system into subprocesses such as:

- Search Train
- Book Ticket
- Cancel Ticket
- Process Payment

Level 2 DFD



Further decomposes subprocesses (e.g., "Book Ticket" → Verify Availability → Calculate Fare → Confirm Booking)

Advantages and Disadvantages of DFD

Advantages	Disadvantages
Helps understand how data moves through the system.	Can be time-consuming to create for large systems.
Offers a clear graphical representation for both technical and non-technical users.	Focuses only on data flow — does not show control flow, performance, or UI details.
Breaks the system into smaller processes for better clarity and documentation.	Needs regular updates; diagrams easily become outdated if the system changes.
Useful for system analysis, requirement gathering, and documentation.	Requires proper knowledge to create accurate and meaningful diagrams.

Control Flow

- **Definition:** Control flow refers to the sequence in which individual instructions, statements, or function calls are executed within a program. It dictates the order of operations and decision-making processes, determining the program's path during execution.
- **Implementation:** Control flow is managed through constructs such as loops (for, while), conditionals (if, switch), and function calls. These structures enable the program to make decisions, repeat operations, and branch into different execution paths based on specific conditions.
- **Purpose:** The primary goal of control flow is to manage the execution order of instructions, ensuring that the program behaves as intended under various conditions.

Key Idea-

- **Focus:** Control flow centers on the execution order of instructions.
- **Execution vs. Data Movement:** Control flow dictates "when" and "which" operations execute.
- **Programming Paradigms:** Imperative programming languages (e.g., C, Java) emphasize control flow, requiring explicit instructions for execution order.

Use Cases

- **Control Flow:** Essential in scenarios requiring complex decision-making, iterative processes, and conditional execution. For example, implementing algorithms with multiple conditional branches or loops relies heavily on control flow structures.

Control Flow Graph (CFG)

A Control Flow Graph (CFG**)** is the graphical representation of control flow or computation during the execution of programs or applications. Control flow graphs are mostly used in static analysis as well as compiler applications, as they can accurately represent the flow inside a program unit. The control flow graph was originally developed by *Frances E. Allen.*

Characteristics of Control Flow Graph

- The control flow graph is process-oriented.
- The control flow graph shows all the paths that can be traversed during a program execution.
- A control flow graph is a directed graph.
- Edges in CFG portray control flow paths and the nodes in CFG portray basic blocks.

There exist 2 designated blocks in the Control Flow Graph:

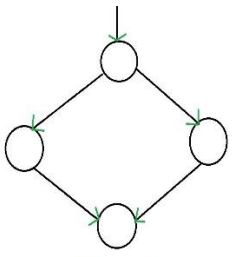
- **Entry Block:** The entry block allows the control to enter into the control flow graph.
- **Exit Block:** Control flow leaves through the exit block.

Hence, the control flow graph comprises all the building blocks involved in a flow diagram such as the start node, end node, and flows between the nodes.

General Control Flow Graphs

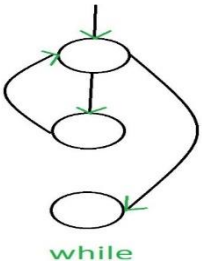
Control Flow Graph is represented differently for all statements and loops. The following images describe it:

1. If-else

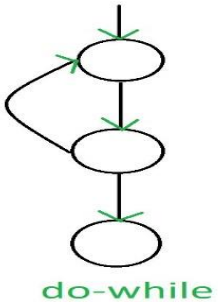


If-then-else

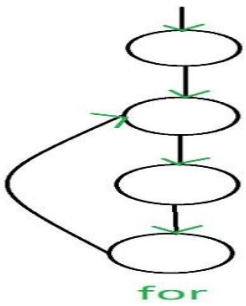
2. While



3. do-while



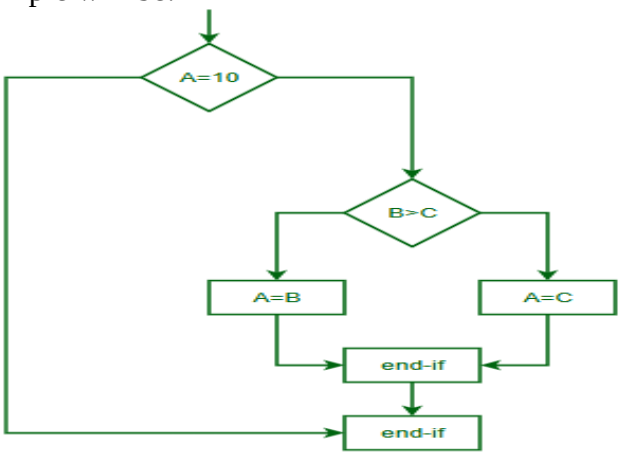
4. for



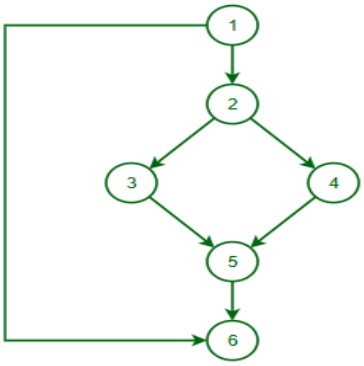
Example

if A = 10 then
if B > C
A = B
else A = C
endif
endif
print A, B, C

Flowchart of above example will be:



Control Flow Graph of above example will be:



Advantage of CFG

- Visualizes program flow: Easy to see how a program runs.
- Helps find errors: Detects unreachable code or infinite loops.
- Useful for optimization: Improves program performance.
- Aids testing: Ensures all parts of the code are tested.

Disadvantages of CFG

- Complex for big programs**:** Hard to understand.
- No unpredictable behaviour**:** Can't show unclear paths.
- No data info: Only shows program flow.
- Not scalable: Becomes messy for large projects.

What is Control Flow?

Control Flow describes the order in which statements, instructions, or function calls are executed in a program.

☞ It determines "what runs next" during program execution.

Without control flow, programs would execute strictly line-by-line with no decision making or repetition.

Types of Control Flow

A. Sequential Control Flow

Default execution order.

- Statements execute one after another
- No branching or looping
- Top → Bottom

Example (C/Java/Python-like):

```
int a = 5;
int b = 10;
int c = a + b;
print(c);
```

Execution order: Line 1 → Line 2 → Line 3 → Line 4

B. Selection (Decision) Control Flow

Allows program to choose between alternatives based on conditions.

(i) if Statement

Executes block only if condition is true.

```
if (x > 0)
   print("Positive");
```

(ii) if–else Statement

Chooses between two paths.

```
if (x % 2 == 0)
   print("Even");
else
   print("Odd");
```

(iii) else-if Ladder

Multiple conditions checked sequentially.

```
if (marks >= 90)
   grade = 'A';
else if (marks >= 75)
   grade = 'B';
else
   grade = 'C';
```

(iv) switch Statement

Used when selecting from many constant values.

```
Switch (day)
{
   case 1: print("Mon"); break;
   case 2: print("Tue"); break;
   default: print("Invalid");
}
```

C. Iteration (Looping) Control Flow

Repeats a block of code multiple times.

(i) for Loop

Used when number of iterations is known.

```
for (int i = 1; i <= 5; i++)
   print(i);
```

(ii) while Loop

Condition checked before execution.

```
while (x > 0)
{
   print(x);
   x--;
}
```

(iii) do-while Loop

Executes at least once (condition checked after).

```
do
{
   print(x);
   x--;
} while (x > 0);
```

D. Jump (Branching) Statements

Alter normal flow abruptly.

break

- Exits loop or switch immediately

continue

- Skips current iteration, moves to next

return

- Exits from function and optionally returns value

3. Function Call Control Flow

When a function is called:

1. Control moves to function definition
2. Function executes
3. Control returns to calling point

main() → functionA() → functionB() → return → main()

Software Requirement Specification (SRS)

In order to form a good SRS, here you will see some points that can be used and should be considered to form a structure of good Software Requirements Specification (SRS). These are below mentioned in the table of contents and are well explained below.

Software Requirement Specification (SRS) Format as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system. These requirements can be functional as well as non-functional depending upon the type of requirement. The interaction between different customers and contractors is done because it is necessary to fully understand the needs of customers.

Depending upon information gathered after interaction, SRS is developed which describes requirements of software that may include changes and modifications that is needed to be done to increase quality of product and to satisfy customer's demand.

Introduction

- **Purpose of this Document -** At first, main aim of why this document is necessary and what's purpose of document is explained and described.
- **Scope of this document -** In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.
- **Overview -** In this, description of product is explained. It's simply summary or overall review of product.

General description

In this, general functions of product which includes objective of user, a user characteristic, features, benefits, about why its importance is mentioned. It also describes features of user community.

Functional Requirements

In this, possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behavior of the system-which outputs should be produced from the given inputs. They describe the relationship between the input and output of the system. For each functional requirement, detailed description all the data inputs and their source, the units of measure, and the range of valid inputs must be specified.

Interface Requirements
In this, software interfaces which mean how software program communicates with each other or users either in form of any language, code, or message are fully described or explained. Examples can be shared memory, data streams, etc.

Performance Requirements
In this, how a software system performs desired functions under specific condition is explained. It also explains required time, required memory, maximum error rate, etc. The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic. Static requirements are those that do not impose constraint on the execution characteristics of the system. Dynamic requirements specify constraints on the execution behaviour of the system.

Design Constraints
In this, constraints which simply means limitation or restriction are specified and explained for design team. Examples may include use of a particular algorithm, hardware and software limitations, etc. There are a number of factors in the client's environment that may restrict the choices of a designer leading to design constraints such factors include standards that must be followed resource limits, operating environment, reliability and security requirements and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.

Non-Functional Attributes
In this, non-functional attributes are explained that are required by software system for better performance. An example may include Security, Portability, Reliability, Reusability, Application compatibility, Data integrity, Scalability capacity, etc.

Preliminary Schedule and Budget
In this, initial version and budget of project plan are explained which include overall time duration required and overall cost required for development of project.

Appendices
In this, additional information like references from where information is gathered, definitions of some specific terms, acronyms, abbreviations, etc. are given and explained.

Uses of SRS document

- Development team require it for developing product according to the need.
- Test plans are generated by testing group based on the describe external behaviour.
- Maintenance and support staff need it to understand what the software product is supposed to do.
- Project manager base their plans and estimates of schedule, effort and resources on it.
- Customer rely on it to know that product they can expect.
- As a contract between developer and customer.
- In documentation purpose.

Importance of Software Requirements Specification (SRS)

SRS (Software Requirement Specification) is a document in which all the user requirements are mentioned in a structural manner. SRS is a kind of agreement between the customer and the company, which contains complete information about the requirement desired by the customer and the product made by the company.

In other words, SRS is a document which describes what will be the features of the software and what will be its behaviour. That is, what kind of performance he will perform. In this article we will see what is SRS, importance of SRS, and characteristics of SRS.

What is Software Requirement Specification (SRS)?

Software requirement specification (SRS) is a complete document or description of the requirements of a system or software application. In other words, "**SRS is a document that describes what the features of the software will be and what its behaviour will be, i.e. how it will perform.**" The advantage of SRS is that it takes less time and effort for developers to develop software. What SRS is is like the layout of the software which the user can review and see whether it (SRS) is made according to his needs or not. We learned about software requirement specification, what it is, now let us read its features.

Importance of Software Requirements Specification (SRS):

1. Clarity and Understanding:
The SRS provides a common communication channel between stakeholders, so that everyone agrees as to what the project aims at. It reduces ambiguity and the risks of misunderstandings among team members in clearly defining requirements.

2. Basis for Project Planning:
The planning for software development projects must be particularly detailed. The SRS is the required information for project managers to define realistic timelines, assign appropriate resources and set milestones. It provides a basis for planning projects, which allows teams to accurately estimate costs and schedules.

3. Guidance for Development Teams:
The SRS is used to guide developers in their understanding of the functional specifications and how the software will be designed. The document offers an overview of the system architecture, user interfaces and data structures which enable developers to build a system that meets with client expectations.

4. Facilitates Testing and Quality Assurance:
SRSs are used as references by quality assurance and testing teams, which use them to design test cases verifying that the software conforms to what was specified. Because testing has well-specified requirements, it becomes much more of a systematic procedure and the chance that an important functionality is overlooked are reduced.

5. Change Management:
During projects, changes are unavoidable. The SRS thus provides a baseline from which any proposed changes can be judged. This guarantees that changes are thoroughly recorded, approved and properly implemented--without scope creep.

6. Client and Stakeholder Satisfaction:
The SRS is a contract between the development team and their client or stakeholders. This boosts client satisfaction and trust if the final product meets what was documented. Moreover, such clear documentation provides a foundation for resolving disputes or disagreements over the course of development.

7. Risk Mitigation:
The earlier in the software development life cycle you can spot potential risks, the better. The SRS helps in risk assessment by specifying dependencies, constraints and possible risks. This makes the development process smoother for project managers who can then come up with strategies to prevent risk.

8. Enhanced Collaboration:
Software development requires collaboration. The SRS has team spirit working together with cross-functional teams, it forms a single vision and shared understanding of the project. As a result, the development process becomes more integrated and efficient.

Characteristics of Software Requirements Specification (SRS):

- **Complete**: The SRS should be complete i.e. all the software requirements should be mentioned in the SRS.
- **Correct**: It should be correct i.e. it should be according to the needs of the customer.
- **Clear**: It should be clear. The requirements of the software should be clearly declared.
- **Accurate**: It should have accuracy. If this is not accurate then the software cannot be developed.
- **Consistent**: It should be consistent from beginning to end so that users can easily understand the requirements. And consistency can be achieved only when there is no contradiction between the two requirements.
- **Verifiable**: It should be verifiable. The requirements are verified by experts and testers.
- **Modifiable**: All the requirements specified in the SRS document should be modifiable. This can happen only when the structure of SRS is balanced.
- **Traceable**: It should be traceable, that is, each requirement in it should be identified differently. Each requirement should have its own identity.
- **Testable**: It should be testable i.e. it should be capable of being tested in any way.
- **Unambiguous**: It can be unambiguous only when all the requirements have only one meaning. That is, there should be only one interpretation.